

ASP.NET Core Web API Main Method

Back to: [ASP.NET Core Web API Tutorials](#)

ASP.NET Core Web API Main Method

The **Main method** is the entry point of an ASP.NET Core Web API project and is responsible for starting the application. It initializes the web host, sets up essential services, and builds the middleware pipeline that processes incoming requests and responses.

What Is the Main Method?

In any C# application, the **entry point** is the Main method. Similarly, in an ASP.NET Core Web API project, the Main method in **Program.cs** serves as the starting point of execution. From here, the web host is created, services are registered, middleware is configured, and finally, the application is run. This method:

- Initializes the **web host**
- Registers required **services**
- Configures the **middleware pipeline**
- Starts listening for HTTP requests with `app.Run()`

In **.NET 6+**, this “Minimal Hosting Model” merges `Program.cs` and `Startup.cs` into a single file for simplicity and readability.

Default Program.cs Class Code:

Let's look at the default `Program.cs` that Visual Studio generates:

```
namespace MyFirstWebAPIProject
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // Add services to the container.
            builder.Services.AddControllers();
            builder.Services.AddEndpointsApiExplorer();
            builder.Services.AddSwaggerGen();

            var app = builder.Build();
```

```
        // Configure the HTTP request pipeline.
        if (app.Environment.IsDevelopment())
        {
            app.UseSwagger();
            app.UseSwaggerUI();
        }

        app.UseHttpsRedirection();

        app.UseAuthorization();

        app.MapControllers();

        app.Run();
    }
}
```

Step-by-Step Execution Flow

Let us understand the Program class code line by line, and also understand what is happening behind the scenes.

Step 1: Host Creation

The first step in creating an ASP.NET Core application is to create the **host**. The host is the container that holds everything your app needs to run: the web server, configuration system, logging, dependency injection, and the middleware pipeline.

When we write: **var builder = WebApplication.CreateBuilder(args);** it creates a **WebApplicationBuilder** object, which sets up the foundation of your application. The returned object (WebApplicationBuilder) gives you access to **services**, **configuration**, and the **environment**.

What Happens Behind the Scenes?

Web Server Setup (Kestrel)

- NET Core apps are hosted on **Kestrel**, a fast, lightweight, and cross-platform web server.
- Kestrel is the default server, so you don't need to configure it explicitly.
- It listens for incoming HTTP requests and forwards them to the ASP.NET Core middleware pipeline.

Configuration System Initialization

- NET Core has a flexible configuration system. By default, it loads settings from multiple sources in this order:

- **json** → Base configuration.
- **{Environment}.json** → Environment-specific overrides (like appsettings.Development.json).
- **User Secrets** → For sensitive keys in local development.
- **Environment Variables** → Useful in Docker, Azure, or production servers.
- **Command-line arguments** → Parameters passed when starting the app.
- All these are merged into a single **IConfiguration object**, which you can inject anywhere in your code.

Logging Setup

- The builder configures default logging providers automatically:
 - **Console Logging** → Logs appear in the terminal/command prompt.
 - **Debug Logging** → Logs appear in Visual Studio's Debug Output.
- Developers can later extend this to include advanced loggers (e.g., Serilog, NLog, Application Insights).

Dependency Injection (DI) Initialization

- NET Core has a built-in **IoC (Inversion of Control) container**.
- The builder prepares an empty **IServiceCollection**.
- You (or the framework) will add services to this collection.
- Later, `builder.Build()` will convert this into a usable **IServiceProvider**, which is the finalized DI container.

Built-in Services Registered at This Stage

Even before you add anything, ASP.NET Core automatically registers several essential services:

- **IConfiguration**: Holds all configuration data from JSON files, env vars, secrets, and CLI args.
- **ILogger<T>**: Provides strongly typed logging for each class. Example: `ILogger<HomeController>` automatically tags logs with the controller's name.
- **IHostEnvironment / IWebHostEnvironment**: Provides information about the environment (Development, Staging, Production). Helpful in loading environment-specific logic.
- **KestrelServer**: The actual server that listens for HTTP requests.

So, when you call **WebApplication.CreateBuilder(args)**, ASP.NET Core:

1. Configures **Kestrel** as the default web server.
2. **Loads Configuration** from multiple sources (appsettings.json, env vars, CLI args, secrets).
3. Sets up **Default Logging Providers** (Console, Debug).
4. Initializes the **DI Container** (`IServiceCollection`).
5. Registers core built-in services, including **IConfiguration**, **ILogger<T>**, **IHostEnvironment**, **IHostApplicationLifetime**, and **Kestrel**, in the DI container.

Step 2: Service Registration in ASP.NET Core

In ASP.NET Core, all features (such as Controllers, Logging, EF Core, Swagger, Authentication, etc.) are registered within the application using **Dependency Injection (DI)**. The `builder.Services` property gives you access to the **IServiceCollection**, where you register these services.

When the app starts, the DI container creates and manages the lifetime of these services, and they can later be injected into controllers, middlewares, or other classes. When you use:

- `builder.Services.AddControllers();`
- `builder.Services.AddEndpointsApiExplorer();`
- `builder.Services.AddSwaggerGen();`

You are telling the framework which features your app needs, and ASP.NET Core internally registers the required services.

builder.Services.AddControllers()

This method registers all services required to use **MVC Controllers** in a Web API. Controllers are central to handling incoming HTTP requests and generating responses. Without this, controllers will not be discovered or invoked.

What happens behind the scenes:

Controller Activation

- Controllers like `WeatherForecastController` are registered so the framework knows they exist.
- Ensures controllers are created via DI.
- Allows you to inject services like `ILogger<T>`, `DbContext`, or custom repositories directly into controllers.

Model Binding

- Maps incoming HTTP data (JSON, query strings, route parameters, form data) into strongly-typed C# objects.
- Example: `{ "Name": "John", "Age": 25 }`
- Automatically mapped to a `Person` object in your action method.

Model Validation

- Validates request data using **Data Annotations** (`[Required]`, `[Range]`, `[EmailAddress]`, etc.).
- If validation fails, the `[ApiController]` attribute automatically returns a **400 Bad Request** with validation error details.

Filters

- Enables filters like **Authorization Filters**, **Action Filters**, and **Exception Filters**.
- Useful for cross-cutting concerns, such as logging, caching, and error handling.

Action Result Execution

- Handles execution of action methods and serialization of return values (e.g., converting C# objects into JSON).
- Applies proper **status codes** automatically (200 OK, 400 Bad Request, etc.).

Note: Without this line, your controllers won't even be part of the application, meaning endpoints like **/WeatherForecast** would return **404 Not Found**.

builder.Services.AddEndpointsApiExplorer()

Adds a service that **discovers API endpoints** exposed by your app. It collects metadata about all the routes defined in your controllers or minimal APIs.

Why it matters:

- Creates metadata about all the API endpoints in your project.
- It is required for **Swagger/OpenAPI generators** to find your endpoints and generate documentation.
- Without this, Swagger UI wouldn't know what routes, parameters, or HTTP methods your API exposes.

Think of this as the **bridge** between your controllers/actions and documentation tools, such as Swagger.

builder.Services.AddSwaggerGen()

This registers all services needed to generate a **Swagger/OpenAPI specification**.

What it does:

- Generates an **OpenAPI specification (swagger.json)** for your API at runtime.
- Provides the backend for **Swagger UI**, where you can test APIs interactively.
- Can be extended to include:
 - XML comments for better documentation.
 - Authentication schemes (e.g., JWT bearer tokens).
 - API versioning.
- Without this, you'd have to manually document and test APIs with external tools like Postman.

Benefits:

- Provides **self-documentation** for your API.
- Developers can try endpoints without using external tools like Postman.
- Helps frontend developers or external teams integrate with your API faster.

By default, this shows your endpoints with their HTTP verbs (GET, POST, PUT, DELETE), parameters, and sample responses.

Step 3: Build the App

Once you finish registering services in the DI container using the **builder.Services**, the next step is to **build the application object**. This is done with:

- **var app = builder.Build();**

This call returns a **WebApplication** instance, representing the actual **ASP.NET Core application** that will run.

What Happens Behind the Scenes?

Dependency Injection (DI) Container is Finalized

- When we register services with the `builder.Services.Add...()`, they are first added to a temporary collection (**IServiceCollection**).
- When you call `Build()`, that temporary service collection (**IServiceCollection**) is **compiled into an IServiceProvider**, which is the actual Dependency Injection (DI) container used at runtime.
- After this point:
 - The service registrations are now locked.
 - You can no longer add new services to the `builder.Services`.
 - The container is ready to resolve dependencies for controllers, middleware, or any class that requests services via constructor injection

Example: When you request an `ILogger<WeatherForecastController>` in your controller constructor, the finalized DI container is what creates an instance of the `ILogger` service and injects it.

Middleware Pipeline is Still Empty

- Middleware defines how HTTP requests and responses are processed (e.g., `UseHttpsRedirection`, `UseAuthorization`).
- At this stage, the app hasn't yet been told **how to handle HTTP requests**.
- This is by design:
 - First, you register services (such as `AddControllers` and `AddSwaggerGen`).

- Then you configure middleware (UseSwagger, UseHttpsRedirection, UseAuthorization, MapControllers, etc.).

Think of it like the **kitchen is ready** (chefs, tools, ingredients are all set up), but the **cooking process (recipes/steps)** hasn't been defined yet.

Host Configuration is Ready

- By this point, the host (the process that will run your web app) is fully configured.
- This includes:
 - **Kestrel** → The web server that listens for HTTP requests.
 - **Configuration system** → Merged from appsettings.json, appsettings.{Environment}.json, secrets, env vars, command-line args.
 - **Logging Providers** → Console, Debug, EventSource, etc.
 - **Environment** → Development, Staging, or Production.

In Short

When you call the **builder.Build()**, you:

- **Finalize DI container** → Services are locked and ready for injection.
- **Have host infrastructure ready** → Kestrel, config, logging, environment.
- **Get a WebApplication instance** → Ready for you to configure middleware and start the app.

This means the infrastructure for your app is complete, but it won't start serving requests until you configure middleware and call the app.Run().

Step 4: Configuring Middleware in ASP.NET Core

In ASP.NET Core, **middleware** are small, pluggable components that process HTTP requests and responses. They are executed in the **order they are registered** in the Program.cs file, forming what's known as the **request pipeline**.

- When a request comes in, it flows **down the pipeline** through each middleware.
- Each middleware can:
 - Perform some work (e.g., logging, authentication).
 - Pass the request to the next middleware (await next() in custom middleware).
 - Or short-circuit the pipeline (e.g., return a 401 Unauthorized).
- The response flows back through the same pipeline in reverse order.

Middleware in ASP.NET Core is a **request delegates** that process HTTP requests and responses. In our project, the middleware configuration occurs after we call `var app = builder.Build();`. Let's look at each middleware registered in the Request Processing Pipeline.

Conditional Swagger Middleware

```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

Environment Check

- Swagger is enabled only if the app is running in **Development mode**.
- This prevents exposing API documentation in **Production**, where it might be a security risk.

app.UseSwagger()

- Generates the **OpenAPI (Swagger) specification** at runtime.
- Creates a JSON file (e.g., `/swagger/v1/swagger.json`) that describes all your endpoints, routes, request/response types, etc.

app.UseSwaggerUI()

- Serves an **interactive Swagger UI webpage** (usually at `/swagger`).
- Allows developers to test endpoints directly from the browser, featuring buttons for GET, POST, PUT, DELETE, and more.

Together, these two middleware components give you **auto-generated API documentation** during development.

HTTPS Redirection Middleware

app.UseHttpsRedirection();

- Forces all **HTTP requests** (`http://`) to be automatically redirected to **HTTPS** (`https://`).
- This ensures **secure communication** by encrypting traffic with SSL/TLS.

Behind the scenes:

- Checks if a request is made via HTTP.
- Responds with an HTTP 307 Temporary Redirect pointing to the HTTPS version of the same URL.

Example:

- Request: `http://localhost:5021/WeatherForecast`
- Redirected to: `https://localhost:7191/WeatherForecast`

This middleware is essential for **security** and is enabled by default in templates. This ensures data (like login credentials or API tokens) is not sent in plain text.

Authorization Middleware

`app.UseAuthorization();`

- Adds the **Authorization Layer** to the pipeline.
- It works with **[Authorize]** attributes placed on controllers or actions.
- If the user does not meet the required policy/role/claim, the request is rejected with a **403 Forbidden** response.

Important Points:

- `UseAuthorization()` only checks permissions.
- It requires **Authentication middleware (e.g., JWT, cookies, Identity)** (e.g., `app.UseAuthentication()`) to run first, which establishes the user's identity.
- In the default template, authentication isn't included — you add it later if your app needs security.
- Without authentication, `UseAuthorization()` alone cannot identify *who* the user is — it can only enforce rules if the user is already authenticated.

Map Controller Routes

`app.MapControllers();`

- Connects **Controller Routes** (defined with `[Route]`, `[HttpGet]`, `[HttpPost]`, etc.) to the middleware pipeline.
- Uses **Attribute Routing** to map requests to the correct controller action.
- Without this, even though controllers exist, their endpoints won't be exposed to the application.

Behind the scenes:

- Registers endpoints with the **Endpoint Routing system**.
- Example: A `ProductsController` with `[HttpGet("api/products")]` becomes an accessible endpoint at: `https://localhost:5001/api/products`

Middleware acts like a **security checkpoint system** — each piece checks or transforms the request before passing it forward.

Step 5: Run the Application

At the end of the Program.cs file, you'll typically see:

- `app.Run();`

This marks the **starting point of your running application**. Everything you configured before (services, middleware, routes, Swagger, etc.) is now locked in place, and the app is ready to handle real HTTP requests.

What Happens When app.Run() is Called?

Application Startup

- The Run() method starts the **Kestrel web server** (the default web server in ASP.NET Core).
- Kestrel begins listening on the URLs defined in configuration (usually found in launchSettings.json or environment variables).
- Example: https://localhost:5001 or http://localhost:5000.

Blocking the Main Thread

- app.Run() is a **blocking call**.
- This means the Main method will not continue after this line.
- The process remains active and continues serving requests until:
 - The application is manually stopped (e.g., pressing **Ctrl+C** in the terminal).
 - The hosting environment shuts it down (e.g., Docker container stop, IIS recycle).

Request Processing Begins

- Once the app is running, incoming HTTP requests hit the **Kestrel server**.
- Kestrel passes the requests into the **middleware pipeline** (app.Use..., app.MapControllers()), which you defined earlier.
- Middleware processes the request step by step until it reaches the matching endpoint (controller action).
- The response flows back through the pipeline and is returned to the client.

Why is app.Run() Important?

- Without it, your application **would never start listening** for requests.
- It's the **entry point for Kestrel's request loop**.
- It guarantees that the middleware pipeline and routing system you configured earlier are put into action.

The **Main method** in ASP.NET Core Web API serves as the foundation of the application, bringing together host creation, service registration, middleware configuration, and request handling into a unified flow. By understanding

its role, developers can better customize the startup process, optimize performance, and build secure, scalable APIs that meet their specific needs.



Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information